第 9 节 实操任务 1：实现 Router 路由模式

> **学习目标**：`python patterns/router.py` 能正确分类并路由 3 种查询类型
> 目标文件：`patterns/router.py`

步骤 1：创建 patterns 目录

```
cd ~/ai-knowledge-base
mkdir -p patterns
touch patterns/__init__.py
```

步骤 2：用 AI 编程工具生成 router.py

以下代码可以用 **OpenCode**、**Claude Code**、**Cursor**、**Trae** 或**通义灵码**等任意 AI 编程工具生成。

提示词：

```
请帮我编写 patterns/router.py，实现 Router 路由模式：

需求：
1. 两层意图分类策略：
   - 第一层：关键词快速匹配（零成本，不调 LLM）
   - 第二层：LLM 分类兜底（处理模糊意图）
2. 三种意图：github_search / knowledge_query / general_chat
3. 每种意图对应一个处理器函数
4. github_search 调用 GitHub Search API (urllib.request)；query 参数必须用
5. knowledge_query 从本地 knowledge/articles/index.json 检索
6. general_chat 调用 LLM 直接回答
7. 统一入口函数 route(query) -> str
8. 包含 if __name__ == "__main__" 测试入口

依赖：使用 workflows/model_client.py 的 chat() 和 chat_json() 函数
chat() 返回 (text, usage) 元组

```

生成的代码：（参考实现）

```
"""Router 模式 — 基于意图分类的请求路由

两层分类策略:
1. 关键词快速匹配 — 零成本，覆盖常见场景
```

```
2. LLM 分类 — 处理模糊意图，确保不漏判
"""

import json
import os
import urllib.parse
import urllib.request
from typing import Callable

from workflows.model_client import chat, chat_json


# --- 处理器定义 ---

def github_search_handler(query: str) -> str:
    """GitHub 搜索处理器：搜索相关仓库并返回摘要"""
    search_query = query.replace("搜索", "").replace("github", "").strip(
    encoded_query = urllib.parse.quote(search_query)  # 处理空格、中文
    url = f"https://api.github.com/search/repositories?q={encoded_query}
    headers = {"Accept": "application/vnd.github.v3+json"}

    token = os.getenv("GITHUB_TOKEN", "")
    if token:
        headers["Authorization"] = f"token {token}"

    try:
        req = urllib.request.Request(url, headers=headers)
        with urllib.request.urlopen(req, timeout=10) as resp:
            data = json.loads(resp.read().decode())

        results = []
        for repo in data.get("items", []):
            results.append(
                f"- [{repo['full_name']}]({repo['html_url']}) "
                f"⭐{repo['stargazers_count']} — {repo.get('description'
            )
        return f"GitHub 搜索结果:\n" + "\n".join(results) if results else
    except Exception as e:
        return f"GitHub 搜索失败: {e}"


def knowledge_query_handler(query: str) -> str:
    """知识库查询处理器：从本地索引检索"""
    base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)
    index_path = os.path.join(base_dir, "knowledge", "articles", "index.

    if not os.path.exists(index_path):
        return "知识库为空，请先运行采集工作流。"
```

```
    with open(index_path, "r", encoding="utf-8") as f:
        index = json.load(f)

    query_lower = query.lower()
    matches = [
        entry for entry in index
        if query_lower in entry.get("title", "").lower()
        or query_lower in entry.get("category", "").lower()
    ]

    if matches:
        lines = [f"- {m['title']} (相关度: {m.get('relevance_score', '?')
        return f"找到 {len(matches)} 条相关知识:\n" + "\n".join(lines)
    return "未找到匹配的知识条目。"


def general_chat_handler(query: str) -> str:
    """通用对话处理器：LLM 直接回答"""
    result, _ = chat(query, system="你是一个专业的 AI 技术顾问。简洁、准确地回
    return result


# --- 路由器核心 ---

HANDLERS: dict[str, Callable[[str], str]] = {
    "github_search": github_search_handler,
    "knowledge_query": knowledge_query_handler,
    "general_chat": general_chat_handler,
}

KEYWORD_RULES: list[tuple[list[str], str]] = [
    (["github", "仓库", "repo", "搜索项目", "trending"], "github_search"),
    (["知识库", "查询", "检索", "已收录", "knowledge"], "knowledge_query"),
]


def classify_intent(query: str) -> str:
    """意图分类：关键词匹配优先，LLM 兜底"""
    query_lower = query.lower()

    # 第一层：关键词匹配（零成本）
    for keywords, intent in KEYWORD_RULES:
        if any(kw in query_lower for kw in keywords):
            return intent

    # 第二层：LLM 分类
    prompt = f"""请判断以下用户查询的意图类别。
```

```

查询: {query}

可选类别:
- github_search: 想搜索 GitHub 上的项目
- knowledge_query: 想查询已有的知识库内容
- general_chat: 一般性技术问题

请只返回类别名称。"""

    result, _ = chat(prompt, system="你是意图分类器。只返回类别名称。", max_t
    intent = result.strip().lower()
    return intent if intent in HANDLERS else "general_chat"


def route(query: str) -> str:
    """路由器入口：分类意图并调用对应处理器"""
    intent = classify_intent(query)
    print(f"[Router] 意图: {intent}")
    return HANDLERS[intent](query)


# --- 测试入口 ---
if __name__ == "__main__":
    import sys
    query = " ".join(sys.argv[1:]) if len(sys.argv) > 1 else "搜索最近的 A
    print(f"查询: {query}\n")
    print(route(query))
```

步骤 3：理解代码

如果你对这段代码有疑问，可以让 AI 编程工具解释：

> `请解释 patterns/router.py 的设计：`
> `1. 为什么用两层分类而不是直接 LLM 分类？`
> `2. KEYWORD_RULES 的数据结构为什么是 list[tuple] 而不是 dict？`
> `3. classify_intent 的兜底逻辑是怎么工作的？`
> `4. 如果要新增一种意图（比如 arxiv_search），需要改哪几处？`

关键设计解读：

设计点	为什么这样做
两层分类	关键词匹配零成本覆盖 80%，LLM 兜底处理剩余 20%
HANDLERS 字典	新增意图只需加一个处理器函数 + 注册，不改路由逻辑
`max_tokens=50`	分类只需返回一个词，限制输出减少成本
兜底 `general_chat`	未识别的意图不报错，降级到通用对话

步骤 4：运行测试

```
cd ~/ai-knowledge-base/v3-multi-agent

# 用 -m 模式从项目根加载（这样 patterns/ 内部的 from workflows import 才能解析
python3 -m patterns.router "搜索最近的 AI Agent 框架"
python3 -m patterns.router "知识库里有什么关于 RAG 的内容"
python3 -m patterns.router "LangGraph 和 CrewAI 有什么区别"

```

期望输出：

```
查询: 搜索最近的 AI Agent 框架
[Router] 意图: github_search
GitHub 搜索结果:
- [langchain-ai/langgraph](...) ⭐12000 — ...
```

步骤 5：提交到 Git

```
git add patterns/__init__.py patterns/router.py
git commit -m "feat: add Router pattern with keyword + LLM classificatio
```

完成！ Router 能根据用户查询自动分发到对应处理器。关键词匹配零成本覆盖常见场景，LLM 兜底处理模糊意图。